

MeshPropagation

RF Propagation & Shadow Mapping for NEPAMesh

Complete Setup & Deployment Guide

Passively monitors Meshtastic MQTT data to build RF propagation maps.
Maps dead zones, analyzes terrain shadows, and suggests optimal node placements.
Served via Discord bot and live web dashboard. Zero additional mesh network overhead.

Table of Contents

1. Prerequisites	3
2. Creating a Discord Application & Bot	3
3. Configuring Bot Permissions & Intents	5
4. Inviting the Bot to Your Server	6
5. Configuration (.env File)	7
6. Building & Running with Docker	9
7. Accessing the Web Dashboard	10
8. Discord Slash Commands	11
9. RF Shadow Mapper & Coverage Analysis	13
10. Architecture Overview	15
11. Troubleshooting	16

1. Prerequisites

Required Software

- Docker (version 20.10 or later)
- Docker Compose (version 2.0 or later, included with Docker Desktop)
- A web browser for accessing the dashboard
- A Discord account

Required Access

- Access to a Meshtastic MQTT broker (community or self-hosted)
- A Discord server where you have 'Manage Server' permission
- Network access from the Docker host to the MQTT broker (port 1883)
- A port open on the Docker host for the web dashboard (default: 5000)

Install Docker

If Docker is not already installed, follow the official guide for your OS:

Linux (Debian/Ubuntu):

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
# Log out and back in for group changes to take effect
```

Then verify:

```
docker --version
docker compose version
```

2. Creating a Discord Application & Bot

You need to create a Discord Application to get a bot token. This token allows MeshPropagation to connect to Discord and respond to slash commands.

1. Open the Discord Developer Portal in your browser:

```
https://discord.com/developers/applications
```

2. Click the "New Application" button in the top right corner.
3. Enter a name for your application (e.g., "MeshPropagation" or "NEPAMesh RF Monitor"). Accept the Terms of Service and click "Create".
4. You will be taken to the application settings page. Note the "Application ID" - you may need this later.
5. Click "Bot" in the left sidebar navigation.
6. Under the "Token" section, click "Reset Token". You may be asked to confirm and enter your 2FA code.
7. Copy the token immediately and save it somewhere secure. You will NOT be able to see this token again.

WARNING: Never share your bot token or commit it to version control. Anyone with your token can control your bot.

3. Configuring Bot Permissions & Intents

Bot Permissions

Still on the Bot settings page in the Developer Portal, configure these settings:

Under "Privileged Gateway Intents":

- Message Content Intent: ENABLED (toggle ON)

The bot needs these permissions in your Discord server (configured when generating the invite link):

- Send Messages - Post propagation maps and alerts
- Embed Links - Rich embeds with maps and charts
- Attach Files - PNG charts and map images
- Use Slash Commands - Register and respond to /propagation, /mesh, etc.

OAuth2 Scopes

Navigate to "OAuth2" in the left sidebar. Under "OAuth2 URL Generator", select these scopes:

- bot
- applications.commands

4. Inviting the Bot to Your Server

1. In the Developer Portal, go to "OAuth2" > "URL Generator".
2. Under "Scopes", check: bot, applications.commands
3. Under "Bot Permissions", check: Send Messages, Embed Links, Attach Files, Use Slash Commands.
4. Copy the generated URL at the bottom of the page and paste it into your browser.
5. Select the Discord server you want to add the bot to from the dropdown. You must have "Manage Server" permission on that server.
6. Click "Authorize" and complete any CAPTCHA if prompted.

The bot should now appear in your server's member list (it will show as offline until you start the MeshPropagation container).

Getting Channel & Server IDs

You'll need the channel ID for anomaly alerts and optionally the server (guild) ID:

1. In Discord, go to User Settings > Advanced > Enable Developer Mode.
2. Right-click the channel for alerts > Copy Channel ID. Save this for the .env file.
3. Right-click the server name > Copy Server ID (optional, for faster command sync).

5. Configuration (.env File)

MeshPropagation uses a .env file for all configuration. Copy the example file and edit it with your values:

```
cp .env.example .env
nano .env    # or your preferred editor
```

Required Settings

```
# Your Discord bot token (from Step 2)
DISCORD_TOKEN=your_actual_token_here

# Channel ID for propagation alerts
DISCORD_ALERT_CHANNEL_ID=123456789012345678
```

MQTT Settings

```
MQTT_HOST=mqtt.example.com
MQTT_PORT=1883
MQTT_USER=your_mqtt_user
MQTT_PASS=your_mqtt_password
MQTT_TOPICS=msh/US/2/e/#
```

Web Dashboard

```
WEB_PORT=5000
# Set to your server's public URL for Discord embed links
WEB_BASE_URL=http://your-server-ip:5000
```

Shadow Analysis Settings

```
# Grid cell size in meters (default 100 = 100x100m cells)
GRID_CELL_SIZE_M=100
# Padding around node positions in km
GRID_PADDING_KM=2.0
# Maximum assumed node range in km
MAX_NODE_RANGE_KM=15.0
# How often to recalculate coverage (seconds)
COVERAGE_RECALC_SEC=600
# Shadow score threshold for dead zone detection
SHADOW_THRESHOLD=0.6
# Minimum cells to form a dead zone
MIN_DEAD_ZONE_CELLS=5
```

Elevation API

```
# Open-Topo-Data endpoint (free, no API key needed)
ELEVATION_API_URL=https://api.opentopodata.org/v1/srtm90m
ELEVATION_BATCH_SIZE=100
ELEVATION_RATE_LIMIT_SEC=1.0
```

Optional: Guild ID for Development

Setting DISCORD_GUILD_ID makes slash commands sync instantly to that server (instead of waiting up to 1 hour for global sync). Recommended during setup.

```
DISCORD_GUILD_ID=123456789012345678
```


6. Building & Running with Docker

Quick Start

```
# Clone or copy the meshpropagation directory to your server
cd meshpropagation

# Copy and edit the environment file
cp .env.example .env
nano .env

# Build and start the container
docker compose up -d --build

# Check logs to verify startup
docker compose logs -f
```

What You Should See in Logs

```
meshpropagation | 2026-04-08 12:00:00 [INFO] meshpropagation: Database ready
meshpropagation | 2026-04-08 12:00:00 [INFO] meshpropagation: MQTT subscriber started
meshpropagation | 2026-04-08 12:00:00 [INFO] mqtt.subscriber: MQTT connected with rc=Success
meshpropagation | 2026-04-08 12:00:00 [INFO] weather.fetcher: Weather fetched: 15.2C...
meshpropagation | 2026-04-08 12:00:00 [INFO] meshpropagation: Starting Discord bot...
meshpropagation | 2026-04-08 12:00:01 [INFO] discord_bot.bot: Discord bot logged in as...
```

Managing the Container

```
# Stop the container
docker compose down

# Restart after config change
docker compose down && docker compose up -d --build

# View live logs
docker compose logs -f

# Check container status
docker compose ps
```

Data is persisted in a Docker volume (meshprop-data). Your database survives container rebuilds.

7. Accessing the Web Dashboard

Once the container is running, open your browser to:

```
http://your-server-ip:5000
```

Available Pages

Dashboard /

Overview of the mesh network: active node count, link observations, weather conditions, node table, and activity charts. Stats auto-refresh every 30 seconds.

Live Map /map

Interactive Folium map showing all nodes as colored markers and RF links as colored lines. Green = good SNR, red = poor. Click nodes/links for details. Supports OpenStreetMap and OpenTopoMap layers. Auto-refreshes every 60 seconds.

Node Detail /node/<node_id>

Detailed view of a single node: identity, hardware, battery, position, active links with SNR stats, and battery history chart.

Shadow Map /shadows

Interactive heatmap showing RF shadow coverage. Green = covered, red = shadow. Dead zone outlines, node markers, and placement suggestions overlay the map.

Shadow Dashboard /shadows/dashboard

Coverage stats grid (coverage %, shadow area, dead zone count), coverage breakdown and timeline charts, dead zone table, and top placement suggestions.

Placement Suggestions /suggestions

Ranked table of optimal node placement locations with elevation, shadow reduction, and reasoning. Includes an interactive evaluate form to test arbitrary coordinates.

API Endpoints

JSON data is available for integration with other tools:

GET /api/summary	- Mesh network summary stats
GET /api/nodes	- All known nodes
GET /api/links?hours=24	- Link observations (aggregated)
GET /api/chart/activity	- Network activity chart (PNG)
GET /api/chart/weather	- Weather correlation chart (PNG)
GET /api/shadow/summary	- Coverage stats (% , areas, dead zones)
GET /api/shadow/deadzones	- Active dead zone list
GET /api/shadow/suggestions	- Placement suggestions
GET /api/shadow/snapshots	- Coverage timeline data
GET /api/shadow/evaluate?lat=&lon=	- Evaluate a placement

8. Discord Slash Commands

After the bot starts and syncs commands, these slash commands are available in your Discord server:

/propagation

Shows the current RF propagation map as a PNG image with network stats. Includes a link to the interactive web map.

/history [hours]

Shows network activity over time. Default is 24 hours. Displays a bar chart of link observations per hour.

/weather [days]

Shows weather vs propagation correlation scatter plots. Displays Pearson correlation coefficients for temperature, humidity, and pressure vs SNR. Also shows current weather conditions.

/node <name>

Shows detailed information for a specific node. Accepts short name, long name, or node ID (e.g., /node Leo or /node !abcd1234). Includes battery history chart.

/mesh

Shows a complete mesh network overview: node counts, top links by observation count, weather conditions, and a network map.

/shadows

Shows the RF shadow map overview with coverage percentage, grade, shadow area, dead zone count, and active dead zone list.

/coverage

Shows coverage percentage with letter grade (A-F), area breakdown (total, covered, shadow in mi²), and a coverage breakdown pie chart.

/suggest

Shows top 5 optimal node placement suggestions ranked by shadow reduction. Includes elevation (ft) and expected improvement percentage.

/deadzone <name>

Shows details for a specific dead zone: area, cell count, cause (terrain/distance/mixed), shadow scores, and center coordinates.

/coverage-history [days]

Shows coverage evolution over time with a timeline chart. Reports whether coverage improved, declined, or held stable.

/evaluate <lat> <lon>

Evaluates a proposed node placement location. Reports expected shadow reduction and rates the location from

excellent to no improvement.

Anomaly Alerts

The bot automatically posts alerts to the configured alert channel when it detects unusual propagation events:

- Signal BOOST (ducting) - SNR significantly above historical average
- Signal FADE - SNR significantly below historical average
- New Link - New long-range link (>10km) detected for the first time
- Link Lost - Known active link has gone silent while both nodes are still active

Shadow alerts are also posted automatically:

- New Dead Zone - A new contiguous shadow region has appeared
- Dead Zone Eliminated - A previously active dead zone is no longer detected
- Coverage Drop - Overall coverage has declined by more than 5%

9. RF Shadow Mapper & Coverage Analysis

The RF Shadow Mapper identifies where mesh RF signals DON'T reach. It builds a grid-based coverage model, detects dead zones using connected component analysis, checks terrain line-of-sight via elevation data, and suggests optimal node placements.

How Coverage is Calculated

The analysis area is defined by a bounding box around all node positions plus padding (default 2 mi). This area is divided into grid cells (default ~330 x 330 ft). For each cell, a coverage score is calculated based on the reliability and distance of reachable nodes. A shadow score is the inverse, weighted by observation confidence to prevent labeling unmonitored areas as dead zones.

Dead Zone Detection

Cells with shadow scores above the threshold (default 0.6) are flagged as shadow cells. Connected components of shadow cells are grouped into dead zones using scipy. Zones smaller than the minimum size (default 5 cells) are filtered out. Each zone is auto-named by cardinal direction and distance from the mesh center.

Terrain Analysis

Elevation data is fetched from the Open-Topo-Data API (SRTM 90m resolution, free, no API key required). The system progressively caches elevation data for grid cells. Line-of-sight checks sample elevation along the path between a node and a cell, comparing against a straight line with Fresnel zone margin. Dead zones are classified as terrain-caused, distance-caused, mixed, or unknown.

Node Placement Optimization

A greedy algorithm evaluates candidate positions within shadow cells. Each candidate is scored by how many other shadow cells it would cover, weighted by elevation (hilltops preferred). Nearby candidates are clustered (>1640 ft apart). The top 5 suggestions are saved with reasoning text and expected shadow reduction.

Coverage Timeline

Hourly snapshots record coverage percentage, areas, active node count, and dead zone count. This enables tracking coverage evolution over days and weeks via the /coverage-history command and web dashboard charts.

The coverage pipeline runs automatically every 10 minutes. Elevation data fetches progressively and may take ~40 minutes for a full grid on first run.

10. Architecture Overview

MeshPropagation runs as a single Docker container with multiple internal threads:

MQTT Subscriber

Connects to the Meshtastic MQTT broker and decodes protobuf packets (Position, NodeInfo, Telemetry, NeighborInfo). Writes all data to SQLite. Runs in its own thread via paho-mqtt loop_start().

Weather Fetcher

Polls the Open-Meteo API every 15 minutes for current weather conditions at the mesh network center. Associates weather data with link observations for correlation analysis. Daemon thread.

Anomaly Detector

Checks every 5 minutes for unusual propagation events by comparing recent SNR measurements against historical baselines. Writes events to the database for the alert dispatcher to pick up. Daemon thread.

Coverage Pipeline

Recalculates the coverage grid every 10 minutes: builds grid, computes coverage/shadow scores, detects dead zones via connected components, and runs the placement optimization algorithm. Daemon thread.

Elevation Fetcher

Progressively fetches SRTM elevation data from Open-Topo-Data for grid cells. Rate-limited to 1 request/sec, 100 points/batch. Caches results in SQLite (terrain doesn't change). Daemon thread.

Snapshot Taker

Takes hourly coverage snapshots for timeline tracking. Records coverage %, areas, active node count, and dead zone count. Daemon thread.

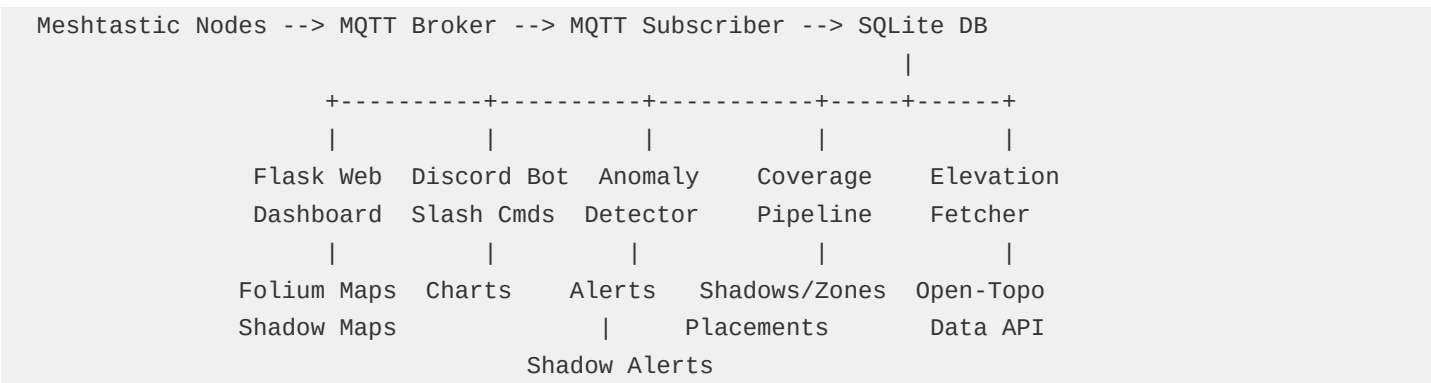
Flask Web Server

Serves the web dashboard, interactive maps, shadow analysis pages, and API endpoints. Generates Folium maps and Matplotlib charts on-demand. Daemon thread.

Discord Bot

Runs discord.py's async event loop on the main thread. Handles slash commands, dispatches anomaly alerts and shadow alerts. Uses asyncio.to_thread() for chart rendering to avoid blocking.

Data Flow



11. Troubleshooting

Problem: Bot is online but slash commands don't appear

Solution: Slash commands can take up to 1 hour to sync globally. Set `DISCORD_GUILD_ID` in `.env` for instant sync to your server. Restart the container after changing this.

Problem: MQTT connection fails

Solution: Check that your Docker host can reach the `MQTT_HOST` on the configured port. Verify `MQTT_USER` and `MQTT_PASS` in `.env`. Check logs for the specific error code.

Problem: No data appearing on the map

Solution: The map requires nodes to report both Position AND NeighborInfo packets. Position data is frequent, but NeighborInfo may take 30+ minutes to appear. Check the dashboard node table - if nodes appear but the map is empty, wait for more data.

Problem: Weather correlation charts show 'No data'

Solution: Correlation requires link observations that have associated weather data. This takes time to accumulate - the weather fetcher runs every 15 minutes and link observations must arrive while weather data exists. Allow at least 24 hours of operation.

Problem: Container exits immediately

Solution: Check logs with 'docker compose logs'. Common causes: invalid `DISCORD_TOKEN`, missing `.env` file, or port conflict on `WEB_PORT`.

Problem: Web dashboard loads but shows no nodes

Solution: MQTT is likely connected but no packets have arrived yet. Check `MQTT_TOPICS` in `.env` matches your mesh network. The default topics (`msh/US/2/e/#` and `msh/nepa/2/e/#`) are correct for NEPAMesh.

Problem: 'Permission denied' errors in Docker

Solution: Ensure the data volume has correct permissions. Try: `docker compose down -v` (warning: deletes data) then `docker compose up -d --build`. Or fix permissions on the volume mount point.

Problem: Charts look wrong or have missing data

Solution: Matplotlib renders are generated on-demand. If data is sparse, charts may look empty. The system needs time to accumulate data - most visualizations improve significantly after 24-48 hours of continuous operation.

Problem: Shadow map shows no data

Solution: The coverage pipeline runs every 10 minutes. On first start, it needs at least 2 nodes with GPS positions before it can build a grid. Check that nodes are reporting positions on the dashboard.

Problem: Elevation data not loading

Solution: The Open-Topo-Data API is rate-limited to 1 request per second with 100 points per request. A full grid may take ~40 minutes. Check logs for 'Elevation fetcher error'. The API may be temporarily unavailable - the fetcher will retry automatically.

Problem: Dead zones not being detected

Solution: Dead zone detection requires enough shadow cells to form connected components above the minimum size (default 5 cells). If your mesh has good coverage, there may genuinely be no dead zones. Try lowering SHADOW_THRESHOLD in .env (default 0.6).

Viewing Logs

```
# All logs
docker compose logs -f

# Filter for MQTT activity
docker compose logs -f | grep mqtt

# Filter for errors only
docker compose logs -f | grep ERROR

# Enable debug logging (in .env)
LOG_LEVEL=DEBUG
```